

mf6_to_flopy

Translate a MODFLOW 6 model folder into a FloPy build script

User & Developer Documentation

1. Overview

mf6_to_flopy is a command-line tool that reads an existing MODFLOW 6 model — a folder containing mfsim.nam and the package input files — and writes a clean, human-readable FloPy build script made of explicit constructor calls. For example, the tool emits code such as:

```
ic = flopy.mf6.ModflowGwfic(gwf, pname="ic", strt=strt)
```

Running the generated script rebuilds the model and calls `sim.write_simulation()`, giving a full round-trip: MF6 files → FloPy code → MF6 files. The script can optionally also run the model and post-process the results (heads, specific discharge, and the List Budget file), turning it into a complete build-run-analyse workflow. This is useful for converting models produced by ModelMuse (or any other pre-processor) into scripted, version-controllable FloPy workflows.

1.1 Why it is general

The tool does not hard-code any package list. It loads the simulation with FloPy and introspects whatever objects, blocks, and datasets are present, then reconstructs them generically. As a result the same code path handles:

- Any model type — GWF, GWT, GWE, PRT.
- Any number of models — single or coupled multi-model simulations.
- Any number of solvers — each IMS registered to exactly the models it solves.
- Exchanges — GWF-GWF, GWF-GWT, GWT-GWT, with the required `exgtype/exgmnamea/exgmnameb`.
- Any grid type — DIS, DISV, DISU.
- Any input package — NPF, STO, IC, RCH(a), EVT(a), WEL, DRN, GHB, RIV, CHD, MAW, SFR, LAK, UZF, MVR, OBS, OC, and transport packages (MST, ADV, DSP, SSM, ...).

2. Installation

The tool requires Python 3.9+ with `numpy` and `flopy`. Optional packages are needed only for specific features.

2.1 Create an isolated environment (recommended)

```
cd C:\Users\xxx.xxx\Downloads\mf6_2_flopy
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

2.2 requirements.txt

```
numpy
flopy
```

```
matplotlib    # --plots / --postprocess
pandas        # --postprocess List Budget tables
geopandas     # --export shapefiles
vtk           # --export --vtk
```

3. Quick start

```
python mf6_to_flopy.py "examples\01_structured_dis" -o build_model.py --postprocess
python build_model.py
```

The first line generates build_model.py from the model folder; the second runs it. With --postprocess the generated script rebuilds the model, runs it, plots heads and specific discharge, and reads the List Budget file.

4. Command-line reference

Usage: python mf6_to_flopy.py <MODEL_FOLDER> [options]

Argument / flag	Default	Description
MODEL_FOLDER	—	Folder containing mfsim.nam and the package files (required).
-o, --out	build_model.py	Path of the FloPy script to write.
--sim-ws	rebuilt	Workspace the generated script writes the rebuilt model into.
--plots	off	Append head-contour plotting code (one PNG per layer).
--export	off	Append grid + attribute shapefile export code.
--vtk	off	With --export, also export the model to VTK (needs vtk).
--keep-external	off	Preserve OPEN/CLOSE external file references instead of inlining data.
--run	off	Append sim.run_simulation() to the generated script.
--postprocess	off	Full workflow: run + heads + specific discharge + List Budget (implies --run/--plots).
--no-sanitize	off	Disable auto-fix of foreign/absolute paths in the name files.

5. Feature details

5.1 Dependency check

On startup the tool verifies that numpy and flopy are importable. If either is missing it prints a one-line install command and exits cleanly. Optional packages (matplotlib, pandas, geopandas, vtk) only trigger a non-fatal note when the corresponding flag is used.

5.2 Plots (--plots)

Adds a block that reads simulated heads via `model.output.head()` and writes a contoured map per layer to `<sim_ws>/head_layer_<k>.png`. Wrapped in `try/except` so it is skipped gracefully when the model has not been run.

5.3 Full workflow (--postprocess)

This is the flagship option. It appends a complete build → run → analyse workflow after the model construction, mirroring a standard FloPy tutorial:

1. Run — calls `sim.run_simulation()` with a success check that raises if MODFLOW 6 does not terminate normally.
2. Heads — reads `gwf.output.head()` and writes a contoured map per layer (`head_layer_*.png`).
3. Specific discharge — computes vectors with `flopy.utils.postprocessing.get_specific_discharge(...)` and plots them (`specific_discharge.png`).
4. List Budget — reads the `.lst` file with `flopy.utils.Mf6ListBudget`, prints the incremental & cumulative budgets, and saves `budget_incremental.csv` / `budget_cumulative.csv`.

Auto-OC: if the source GWF model has no Output Control package, `--postprocess` automatically adds one (saverecord HEAD ALL + BUDGET ALL) so heads and budget are recorded. `--postprocess` implies `--run` and `--plots`, and every step is wrapped in `try/except` so a missing mf6 executable or optional dependency degrades gracefully.

5.4 GIS export (--export, --vtk)

With `--export` the generated script writes a `<sim_ws>/gis/` folder containing, for each model: a grid shapefile (`<model>_grid.shp`), a package-attribute shapefile (`<model>_attrs.shp`), and — with `--vtk` — VTK output for ParaView.

5.5 External file references (--keep-external)

MODFLOW 6 arrays are often stored in separate files via OPEN/CLOSE. By default the tool inlines those arrays as literal Python values (numerically exact). With `--keep-external` it emits a factory record that preserves the reference, e.g. `k={'filename': 'k_external.txt'}`.

5.6 Foreign/absolute path repair (automatic; --no-sanitize to disable)

Models exported by ModelMuse or GMS frequently embed absolute paths from the build machine, e.g. `TDIS6 C:/Users/Someone/.../Model1.tdis`. The tool scans the name files and, if it finds a reference whose basename exists locally, rewrites it to the plain basename on a temporary copy of the folder — your originals are never modified. Pass `--no-sanitize` to disable. A companion script, `fix_mf6_paths.py`, performs the same repair permanently (with a backup).

6. How it works

1. Load the simulation with `flopy.mf6.MFSimulation.load()` (after auto-sanitizing foreign paths if needed).
2. Emit the `MFSimulation` constructor, then every simulation-level package (TDIS, IMS, ...), skipping exchanges.

3. Register each IMS solver with the models it solves via `sim.register_ims_package(...)`.
4. For each model, emit the model constructor and one constructor per package (adding OC when `--postprocess` needs it).
5. Emit exchange packages with `exgtype` / `exgmnamea` / `exgmnameb`.
6. Finish with `sim.write_simulation()` and any optional run / plot / post-process / export blocks.

6.1 Data-cleaning rules

Source value	Emitted as
Array constant everywhere	scalar (e.g. <code>top=50</code>)
3-D array constant per layer	per-layer list (e.g. <code>botm=[40, 35, 0]</code>)
Any other array	nested Python list
Keyword records (auxiliary, newtonoptions)	flattened list with the repeated keyword / flag stripped
Period / transient data	{kper: value} dict keyed by integer period, empties dropped
Label-keyed data (e.g. OBS continuous)	dict keyed by the label (output filename)
Cell-id fields (cellid, OBS id/id2)	(layer, row, col) tuples
Unset optional columns (aux/boundname = NaN)	omitted
File-record options (head_filerecord, ...)	scalar filename string
Auto dimensions (maxbound, nbound)	omitted — FloPy recomputes them

7. Programmatic use and the notebook

Every CLI flag maps to a keyword argument on `generate_script()`:

```
import mf6_to_flopy as m2f
m2f.check_dependencies(need_plots=True, need_postprocess=True)
m2f.generate_script(
    'examples/01_structured_dis',
    out_path='build_model.py',
    sim_ws='rebuilt',
    add_postprocess=True,    # run + heads + specific discharge + List Budget
)
```

A ready-to-run tutorial notebook is provided at `notebooks/mf6_to_flopy_tutorial1.ipynb`. It walks through the dependency check, code generation, inspection, round-trip rebuild, verification, the full post-processing workflow, and the optional GIS steps.

7.1 Example set

The project ships four ready-to-use MF6 models in `examples/`:

- `examples/01_structured_dis` — a structured DIS grid with OC (the notebook default).
- `examples/02_unstructured_disv` — an unstructured DISV (vertex) grid.

- `examples/03_unstructured_disu` — a fully unstructured DISU (node-connectivity) grid.
- `examples/04_coupled_gwf_gwt` — a coupled two-model GWF-GWT simulation (flow + solute transport).

7.2 Running the notebook tutorial

1. Activate the environment and install Jupyter: `python -m pip install jupyter`.
2. From the project root, launch Jupyter: `jupyter notebook` (or `jupyter lab`).
3. Open `notebooks/mf6_to_flopy_tutorial.ipynb`.
4. Run the cells top to bottom (Kernel → Restart & Run All). No edits are needed — it points at `examples/01_structured_dis` by default.
5. To convert your own model, change the `model_folder` variable in step 2 to your MF6 folder path and re-run.

8. Validation and known behaviour

The tool was validated by generating code from several models, running that code, and comparing the rebuilt input files against the originals:

Test model	Contents	Result
Simple GWF	DIS, IC, NPF, RCH, RIV/CHD, OC	All packages identical
Coupled GWF+GWT	2 models, shared IMS, GWF-GWT exchange, STO, WEL aux+boundnames, DRN/GHB/RIV, SSM, MST/ADV/DSP	All packages identical
DISV (unstructured)	vertices/cell2d grid, CHD, NPF, OC	All packages identical
DISU (fully unstructured)	node connectivity (<code>iac/ja/ihc/cl12/hwva</code>), CHD, NPF, OC	All arrays identical (diff 0.0)
Observations (OBS)	continuous obs keyed by output CSV filename, cellid tuples	OBS file identical
External arrays	NPF k stored via OPEN/CLOSE	Inline: exact; --keep-external: reference preserved
Foreign paths	absolute <code>C:/Users/...</code> paths in name files	Auto-sanitized; loads and rebuilds; originals untouched
Post-process workflow	--postprocess: run, heads, specific discharge, List Budget	Generated script compiles and runs; APIs verified

Known cosmetic differences: numeric formatting (e.g. 50 vs 5.0000E+01) and keyword case (head vs HEAD). MF6 treats these identically.

8.1 Limitations

- Running the model (and therefore --run/--postprocess output) requires the mf6 executable on PATH; without it those steps are skipped gracefully.
- Time-series (TS) and time-array-series (TAS) helper files are reconstructed from the values FloPy exposes; exotic options not surfaced by FloPy are not emitted.
- VTK export requires the vtk package, which is not always available as a pre-built wheel on the newest Python releases.